CHAPTER 1: Introduction to AI and Intelligent Systems

Understand the field before choosing algorithms.

Artificial Intelligence (AI) is the discipline of building systems that perform tasks associated with human intelligence, including perception, language understanding, reasoning, planning, and decision support. AI may use rules, search, optimization, machine learning, or combinations of these.

1.1 AI, ML, Deep Learning, and Generative AI

- **Artificial Intelligence:** The umbrella field for systems that exhibit intelligent behaviour.
- **Machine Learning (ML):** A branch of AI in which models learn patterns from data rather than relying only on manually written rules.
- **Deep Learning (DL):** ML using neural networks with multiple layers; useful for complex image, audio, and text patterns.
- **Generative AI:** Models that generate content such as text, images, audio, code, or video.

1.2 Types and applications

- **Narrow AI:** Built for a particular task; most deployed AI today fits this category. **General AI** remains hypothetical as a broadly human-level system.
- **Rule-based systems** apply explicit conditions; **learning-based systems** infer patterns from examples.
- Applications include fraud detection, recommendation, demand forecasting, medical-image support, translation, document processing, predictive maintenance, and assistants.

Key idea: AI is the overall field; ML is one approach within AI; deep learning is a subset of ML; generative AI focuses on creating content.

CHAPTER 2: Python, Data, and Mathematics Foundations

The practical toolkit behind most AI/ML workflows.

2.1 Python ecosystem

- **NumPy:** Arrays and numerical operations. **pandas:** Tabular data cleaning, joins, grouping, and filtering.
- **Matplotlib / Seaborn:** Visual exploration. **scikit-learn:** Classical ML, preprocessing, pipelines, and metrics.
- **PyTorch / TensorFlow:** Neural networks. **Jupyter:** Interactive, reproducible experiments.

2.2 Mathematics essentials

- **Linear algebra:** Vectors represent feature rows or embeddings; matrices represent datasets and transformations.
- **Statistics:** Mean, median, variance, probability, distributions, sampling, and confidence intervals describe data and uncertainty.
- **Calculus:** Gradients describe how loss changes with parameters; gradient descent uses them to update parameters.
- **Optimization:** Training seeks parameters that minimize a loss function under practical constraints.

2.3 Essential vocabulary and workflow

Feature: Input variable. **Label/target:** Value to predict. **Parameter:** Learned value. **Hyperparameter:** Setting chosen around training. **Inference:** Using a trained model. A basic workflow: load data, inspect types and missing values, explore distributions, split the data, fit preprocessing on training data, train a baseline, and record metrics.

Practice: Load a CSV, print rows and columns, identify numeric/categorical fields, and plot one feature.

CHAPTER 3: Machine Learning Workflow and Data Preparation

A reliable process matters as much as the algorithm.

3.1 End-to-end workflow

- **Define:** Decision, target, success metric, constraints, and baseline.
- **Collect:** Confirm source, permissions, time range, and what each row represents.
- **Explore:** Inspect distributions, imbalance, outliers, missing values, correlations, and leakage.
- **Prepare:** Clean, encode categories, scale when needed, engineer features, and split correctly.
- **Train and tune:** Fit candidates using training data and validation/cross-validation.
- **Evaluate and deploy:** Measure real-world-relevant errors; monitor quality, latency, and drift.

3.2 Data preparation

- **Missing values:** Investigate why they are missing; impute appropriately and consider indicators.
- **Categorical encoding:** One-hot encoding suits many nominal fields; ordinal encoding requires meaningful order.
- **Scaling:** Often helps distance-based and gradient-based models.
- **Outliers:** Verify whether values are errors or genuine rare cases before changing them.
- **Imbalance:** Use stratified splits, suitable metrics, class weights, or resampling where justified.

3.3 Data leakage and splitting

Leakage occurs when training uses information unavailable at prediction time—for example, future outcomes or preprocessing fitted on the full dataset. Use chronological splits for time-dependent data and group-based splits when several rows belong to the same person or entity. Fit transformations on training data only and do not repeatedly tune against the test set.

Rule: The test set is for final evaluation, not repeated model selection.

CHAPTER 4: Supervised Learning: Regression and Classification

Learn from examples with known target values.

Supervised learning uses labelled examples. **Regression** predicts numeric values; **classification** predicts classes or categories.

4.1 Common algorithms

- **Linear Regression:** Predicts numeric targets from weighted features; a useful baseline.
- **Logistic Regression:** Estimates class probabilities, often for binary classification.
- **Decision Tree:** Splits data by features; interpretable but may overfit when deep.
- **Random Forest:** Combines many randomized trees to improve robustness.
- **Gradient Boosting:** Builds models sequentially to correct earlier errors; examples include XGBoost, LightGBM, and CatBoost.
- **SVM:** Finds a large-margin boundary; kernels support nonlinear boundaries.
- **KNN:** Predicts from nearby examples and is sensitive to scaling.
- **Naive Bayes:** Uses Bayes' theorem with conditional-independence assumptions; useful for text baselines.

4.2 Metrics

Task	Metrics	Interpretation
Regression	MAE, RMSE, R^2	Error magnitude and fit versus a mean-target baseline.
Classification	Accuracy, precision, recall, F1, ROC-AUC	Different views of correct predictions and error trade-offs.

4.3 Underfitting and overfitting

Underfitting means the model is too simple to capture patterns; overfitting means it learns training noise and generalizes poorly. Validation, regularization, cross-validation, simpler models, representative data, and early stopping can help.

Example: For screening, recall may matter because missed positives are costly, while precision matters because false alarms create unnecessary follow-up.

CHAPTER 5: Unsupervised Learning and Dimensionality Reduction

Find structure when labels are unavailable.

Unsupervised learning finds structure without a target label. Uses include segmentation, anomaly exploration, compression, and discovering patterns for later analysis.

5.1 Clustering

- **K-Means:** Assigns observations to K centroid-based clusters; requires K and can be affected by scaling and outliers.
- **Hierarchical clustering:** Builds nested groups, often inspected using a dendrogram.
- **DBSCAN:** Groups dense regions and labels sparse points as noise; detects irregular shapes but depends on density settings.

5.2 Dimensionality reduction

- **PCA:** Creates orthogonal components that capture variance; useful for compression, though components may be hard to interpret.
- **t-SNE:** Visualizes local neighbourhoods; do not over-interpret distances between separated groups.
- **UMAP:** Common for low-dimensional visualization; results depend on parameters and sampling.

5.3 Anomaly detection and evaluation

Methods include Isolation Forest, Local Outlier Factor, statistical thresholds, and autoencoder reconstruction error. An anomaly is not automatically fraud or an error. Without labels, inspect cluster stability, silhouette score, domain usefulness, and whether groups support a real decision; do not assume every dataset has natural clusters.

Practice: Scale numeric data, compare K-Means across several K values using silhouette scores, and plot two principal components. Explain what the plot can and cannot prove.

CHAPTER 6: Deep Learning and Neural Networks

Learn representations through layers of trainable operations.

A neural network transforms inputs through layers of weighted sums and nonlinear activations. Training uses a loss function, backpropagation to calculate gradients, and an optimizer to update weights.

6.1 Core components

- **Neuron:** Combines inputs using weights and bias, then applies an activation.
- **Activations:** ReLU is common in hidden layers; sigmoid can represent binary probability; softmax converts class scores to probabilities.
- **Loss:** Measures error, e.g. mean squared error or cross-entropy.
- **Backpropagation:** Uses the chain rule to calculate gradients.
- **Optimizer:** SGD and Adam update parameters. An epoch is a pass through data; a batch is a subset; learning rate controls update size.

6.2 Architectures

- **MLP:** Feed-forward network for vector or tabular inputs.
- **CNN:** Learns spatial features, especially in images.
- **RNN/LSTM/GRU:** Designed for sequential data; transformers dominate many modern language tasks.
- **Transformer:** Uses attention to model relationships between tokens and supports parallel training.
- **Autoencoder:** Learns compressed representations and reconstructs inputs.

6.3 Training challenges

Networks can overfit, require compute, and be sensitive to hyperparameters. Dropout, weight decay, augmentation, early stopping, normalization, and learning-rate schedules can help. Compare training and validation loss; a widening gap can indicate overfitting. Tree-based methods often remain strong for tabular data.

Practical note: Start with a baseline and small model; use a GPU when it materially improves training speed.

CHAPTER 7: Natural Language Processing (NLP)

Turn text into useful representations and predictions.

NLP enables systems to process language for sentiment analysis, classification, information extraction, translation, summarization, question answering, and text generation.

7.1 Preprocessing

- **Normalization:** Standardize text when useful; preserve case and punctuation if meaningful.
- **Tokenization:** Split text into words, subwords, or characters; modern LLMs commonly use subwords.
- **Stop words:** Removing frequent words is task-dependent and can damage meaning.
- **Stemming:** Heuristically reduces words to stems, which may not be valid words.
- **Lemmatization:** Maps words to dictionary forms using linguistic information.
- **NER:** Detects entities such as people, organizations, dates, and locations.

7.2 Representations and transformers

- **Bag of Words:** Token counts, largely ignoring word order. **TF-IDF:** Weights informative terms.
- **Embeddings:** Dense vectors that encode learned patterns; contextual embeddings change with surrounding text.
- **Transformer:** Uses attention to combine sequence information. An LLM commonly predicts the next token, then may be instruction-tuned.
- **Token:** A unit of text processed by the model. **Context window:** The input and generated context the model can consider in one request.

7.3 Project workflow

For sentiment classification, define labels, inspect imbalance, split carefully, build a TF-IDF plus Logistic Regression baseline, evaluate precision/recall/F1, inspect errors, then compare with a transformer if justified. Protect private text and test different writing styles.

Key distinction: Stemming and lemmatization are not mandatory for every NLP pipeline, especially with pretrained transformers.

CHAPTER 8: Computer Vision and Image Intelligence

Extract meaning from images and video.

Computer Vision interprets visual data for defect inspection, medical-image support, object tracking, document understanding, image search, and scene analysis.

8.1 Main tasks

- **Classification:** Assigns labels to an image. **Detection:** Predicts object classes and bounding boxes.
- **Segmentation:** Labels pixels or regions; instance segmentation separates individual objects.
- **OCR:** Extracts text from images or scanned documents. **Tracking:** Associates objects across video frames.

8.2 Typical pipeline

- Collect representative images with appropriate permissions; annotate labels, boxes, masks, or text.
- Resize and normalize consistently; augment training images appropriately.
- Use transfer learning from a pretrained model when suitable.
- Evaluate on a held-out set reflecting deployment conditions and inspect errors by class, lighting, camera, and image quality.

8.3 Transfer learning and risks

Transfer learning reuses features learned from a large dataset. Replace the final layer, train the new head, then optionally fine-tune the backbone. It can reduce data and compute needs, but domain mismatch can limit performance. Systems can fail under blur, occlusion, unusual angles, poor lighting, or distribution shift. High-impact use requires privacy, consent, human review, and careful error analysis.

Practice: Train a pretrained CNN classifier, apply augmentation to training data only, compare training and validation curves, and document misclassified examples.

CHAPTER 9: Generative AI, LLMs, RAG, and Vector Search

From language generation to grounded retrieval systems.

Generative models learn patterns and create new outputs. LLMs generate text token by token from prompts and prior context; fluent output can still be incorrect, so important claims need verification.

9.1 Prompting and model settings

- **Prompt:** Instructions and context guiding the model; clear constraints and examples improve consistency.
- **Temperature:** Controls sampling randomness but does not guarantee correctness.
- **Hallucination:** Unsupported or false output stated confidently. **Fine-tuning:** Updates parameters using task-specific examples; prompting does not.

9.2 Retrieval-Augmented Generation (RAG)

RAG combines retrieval with generation. Typical flow: ingest documents → extract text → chunk → create embeddings → store vectors and metadata → retrieve relevant passages → optionally rerank → place context in the prompt → generate an answer with source references.

9.3 Embeddings, vector search, and quality

Embeddings are numeric vectors representing learned features. Vector search retrieves nearby vectors using cosine similarity or Euclidean distance. FAISS is a library for efficient similarity search, not by itself a complete managed database. Tune chunk size and overlap; evaluate top-k retrieval relevance, answer correctness, citation support, latency, and cost. Consider hybrid keyword/vector search and reranking.

9.4 Security

Treat retrieved documents as untrusted input. Guard against prompt injection, access-control failures, and sensitive-data exposure. Instruct the model to use retrieved evidence and admit when evidence is insufficient.

Example: A policy assistant retrieves approved passages, answers from them, and cites the source. It should not invent a policy when retrieval fails.

CHAPTER 10: Evaluation, Responsible AI, Deployment, and Projects

Turn a prototype into a trustworthy system.

10.1 Evaluation and monitoring

Evaluate on data not used for fitting or model selection. Compare with a baseline, analyse errors by relevant segments, and document limitations. In production, monitor input drift, prediction distributions, latency, failures, and outcome metrics when labels arrive. Performance may degrade as user behaviour or environments change.

10.2 Responsible AI

- **Fairness:** Check performance across relevant groups and consider historical bias in data or labels.
- **Privacy:** Minimize collection, restrict access, protect sensitive fields, and follow applicable rules.
- **Explainability:** Use interpretable models or suitable explanations where decisions must be understood.
- **Robustness and security:** Test unusual inputs, adversarial behaviour, and misuse scenarios.
- **Human oversight:** Keep qualified people involved in high-impact decisions and provide ways to challenge outcomes.

10.3 Deployment lifecycle

Serve models through an API or application, version the model and preprocessing, validate inputs, record operational metrics safely, add tests and access controls, and support rollback. Keep experiments reproducible and document retraining triggers. A successful notebook is not yet a production system.

10.4 Portfolio roadmap

- **Beginner:** EDA plus a regression/classification model with metrics.
- **Intermediate:** Customer segmentation or anomaly detection with validation explained.
- **NLP:** Sentiment analysis comparing TF-IDF with a transformer.
- **Vision:** Transfer-learning image classifier with error analysis.
- **GenAI:** RAG app with chunking, embeddings, FAISS or another vector store, citations, and evaluation questions.

10.5 Interview checklist

Be ready to explain AI vs ML vs DL; data leakage; train/validation/test splits; bias and variance; precision vs recall; cross-validation; scaling; gradient descent; embeddings; transformers; tokens and context windows; RAG vs fine-tuning; vector similarity; hallucinations; deployment monitoring; and how you measured project impact.

Final takeaway: Strong AI/ML work combines sound problem framing, reliable data, appropriate models, honest evaluation, responsible design, and ongoing monitoring—not just a sophisticated algorithm.